

Employee Management System

Application Architecture & Data Flow

This document explains the overall architecture, authentication flow, data flow, role-based functionality, and state management used in the Employee Management System.

1. Application Overview

The Employee Management System is a React application designed around two user roles:

- Admin
- Employee

The application currently uses:

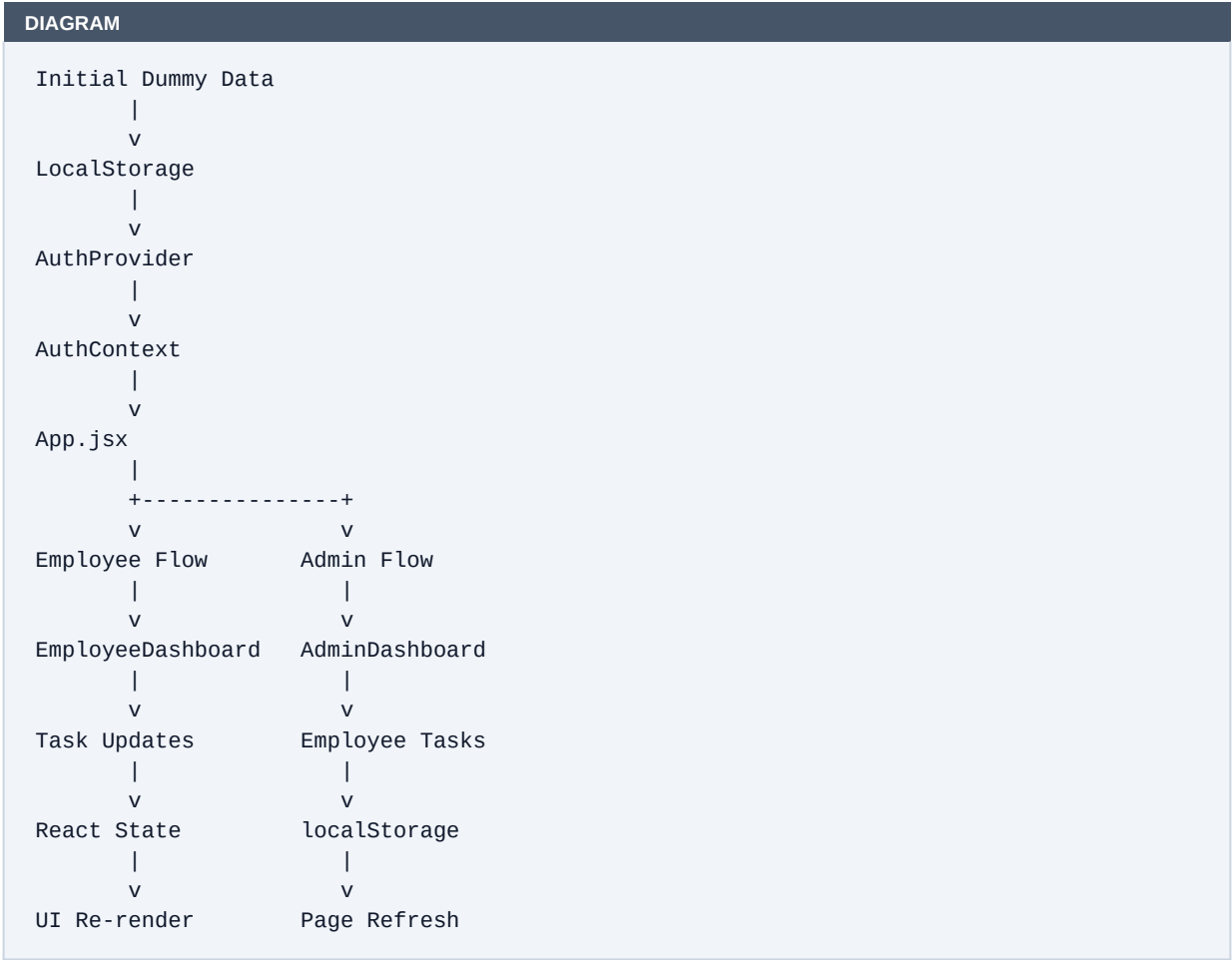
- React
- React Router
- Context API
- React State (`useState`)
- `useEffect`
- `localStorage`
- Tailwind CSS
- Sonner for toast notifications

The core architecture can be understood with one important concept:

`localStorage` is used as a lightweight client-side data storage layer, while React state controls what is currently displayed in the UI.

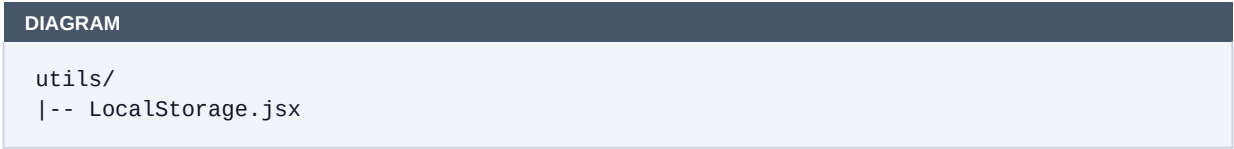
This allows data to persist after a browser refresh while React handles dynamic UI updates.

2. High-Level Data Flow



3. Initial Application Data

The initial application data is defined inside:



The application maintains two main collections:



The overall data structure looks like:

DIAGRAM

```
localStorage
|
|-- employees
|   |-- Pallavi
|   |   |-- tasks
|   |
|   |-- Priya
|   |   |-- tasks
|   |
|   |-- Arjun
|   |   |-- tasks
|   |
|   |-- Sourav
|   |   |-- tasks
|
|-- admin
    |-- Administrator
```

Employees contain their own task data.

JS

```
{
  id: 4,
  name: "Sourav Sen",
  role: "employee",
  tasks: [...],
  taskCounts: {...}
}
```

The admin object does not contain employee tasks.

JS

```
{
  id: 1,
  name: "Ambar Dhara",
  role: "admin"
}
```

This difference is important because it affects how Employee and Admin data updates are handled.

4. AuthProvider Flow

[AuthProvider](#) is responsible for loading the application's primary data.

The flow works as follows:



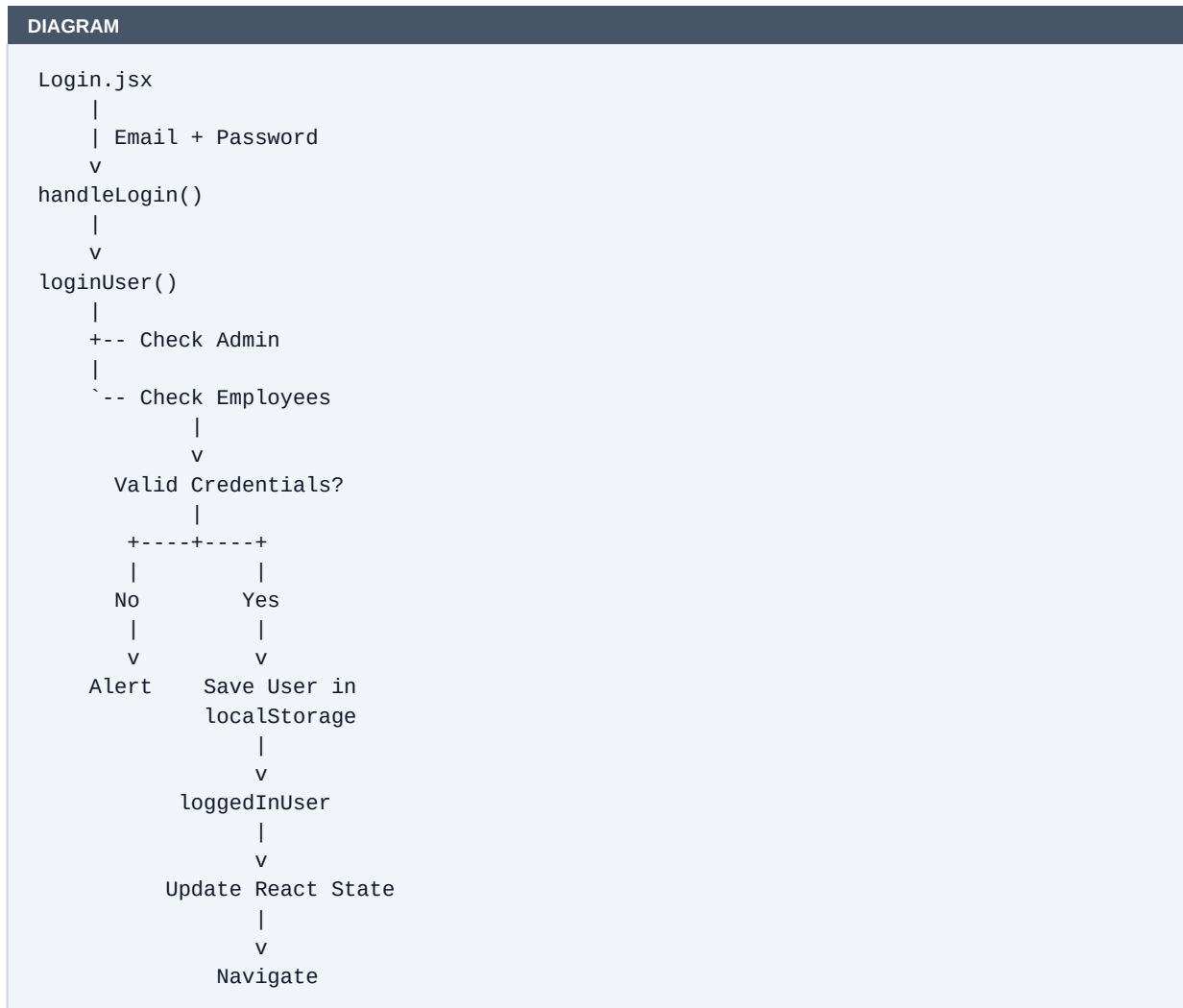
`AuthProvider` provides the following data through Context API:

```
JS
{
  employees: [...],
  admin: [...]
}
```

This allows the application's authentication and user data to be accessed without excessive prop drilling.

5. Login Flow

The login process follows this flow:



After successful authentication, only the essential user identity is stored inside `loggedInUser`.

Example:

JS

```
{
  role: "employee",
  email: "example@gmail.com"
}
```

or:

JS

```
{
  role: "admin",
  email: "admin@gmail.com"
}
```

The complete user information is then restored from the employee or admin data when required.

6. Page Refresh and Session Restoration

React state is temporary.

For example:

JS

```
const [user, setUser] = useState(null);

const [loggedInUserData, setLoggedInUserData] = useState(null);
```

These values are reset when the browser page is refreshed.

To maintain the login session, the application uses:

JS

```
restoreLoggedInUser()
```

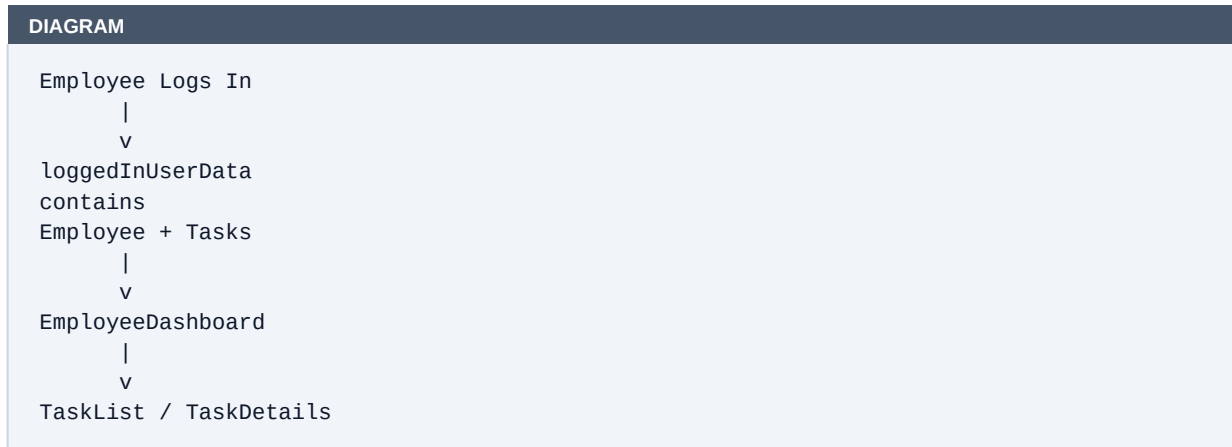
The restoration flow works like this:



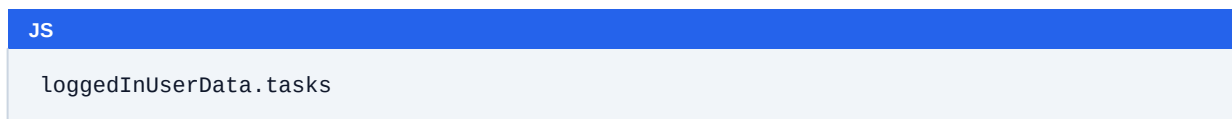
This allows users to remain logged in after refreshing the browser.

7. Employee Data Flow

The employee flow is relatively straightforward because the logged-in employee's tasks are included directly inside `loggedInUserData`.



For example:



contains the currently logged-in employee's tasks.

When a task is updated:



Updating React state causes the UI to automatically re-render.

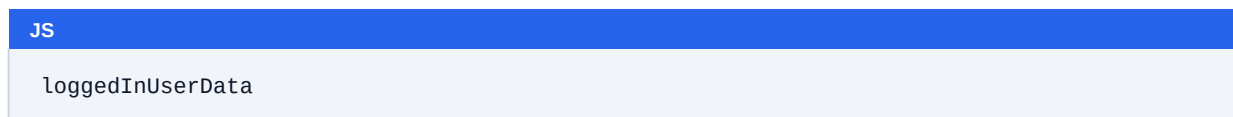
At the same time, updating `localStorage` ensures that the changes remain available after a page refresh.

8. Employee Comment Flow

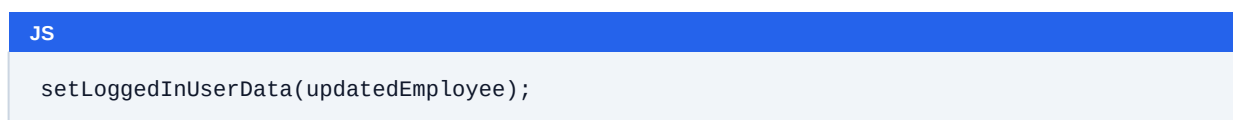
Employee comments follow a similar pattern.



Since the employee's tasks are already available inside:



the application can update React state directly:



React detects the state change and automatically updates the UI.

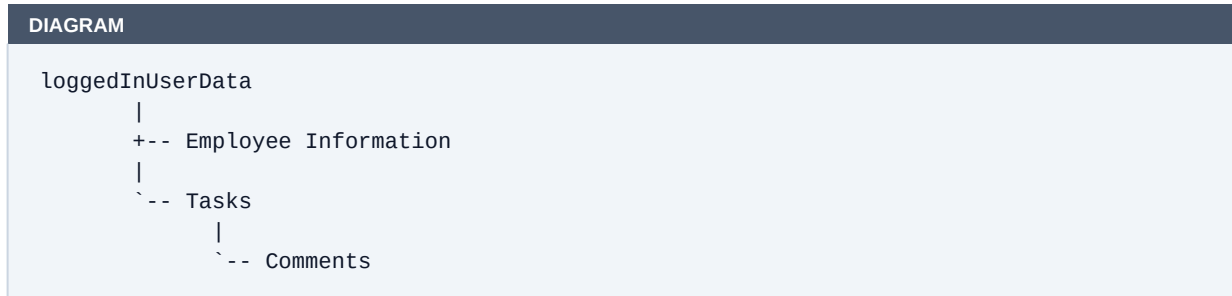
Therefore, no page refresh is required for employee comments.

9. Why Admin Needs a Refresh but Employee Doesn't

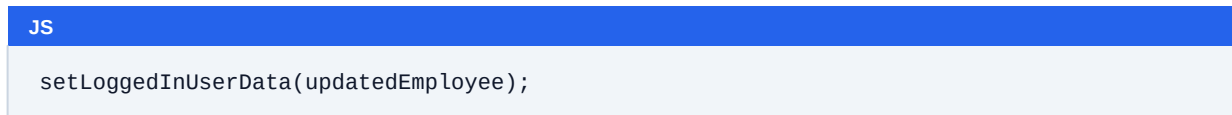
This is an important difference in the current data architecture.

Employee

The employee's data structure is:



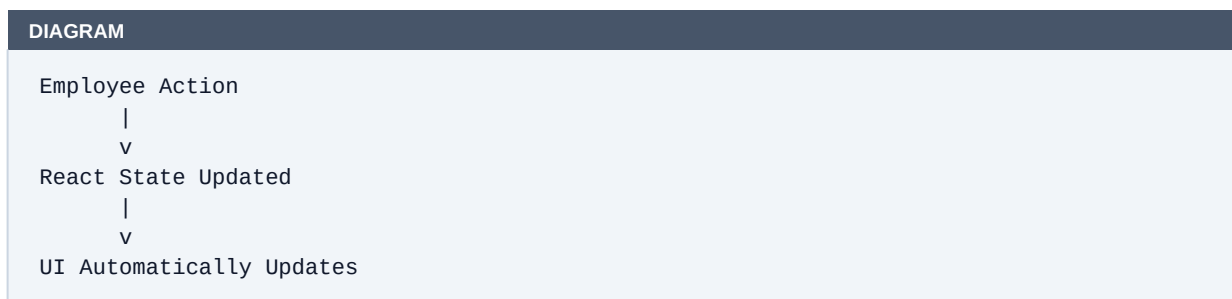
When an employee adds or deletes a comment:



the React state changes.

React automatically re-renders the UI.

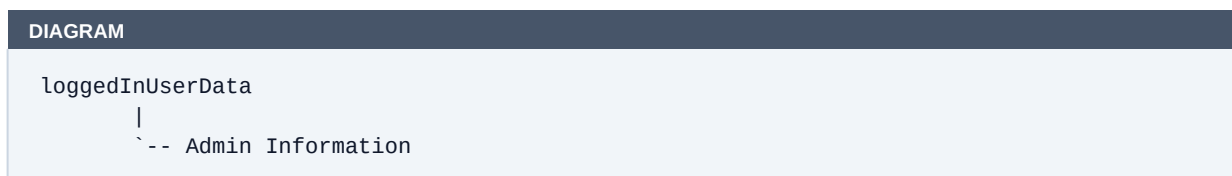
Therefore:



No page refresh is required.

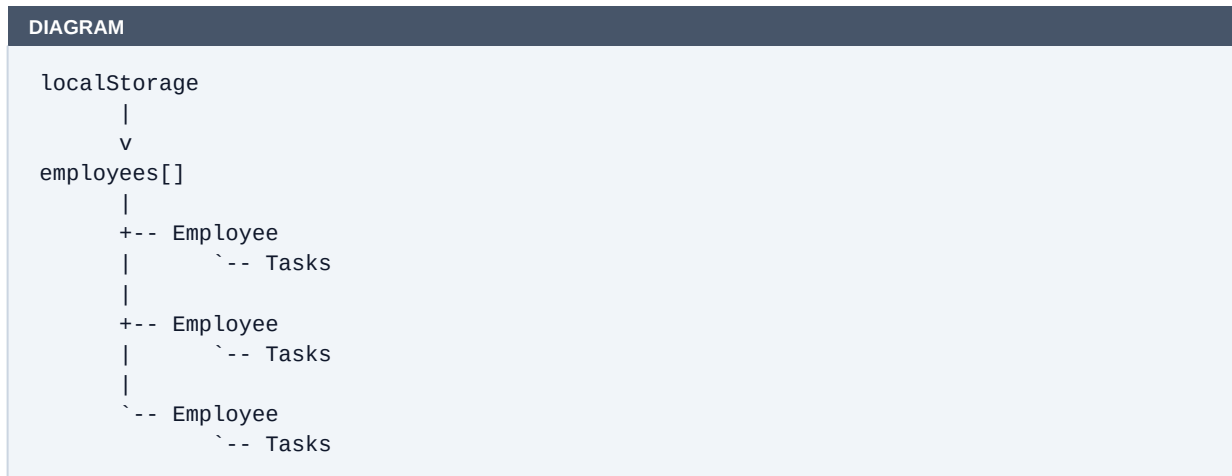
Admin

The admin's data structure is different.

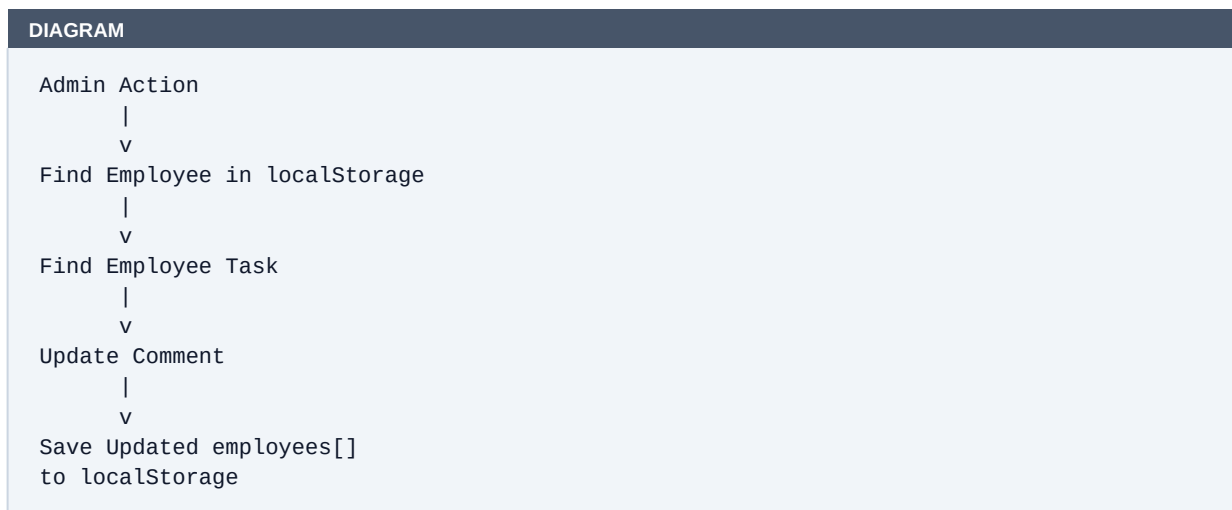


The admin object does not contain employee tasks.

Employee tasks are stored separately inside:



When an admin adds or deletes a comment:



The admin task component ([AllTask](#)) maintains its own employee state:

JS

```

const [employees, setEmployees] = useState([]);
  
```

The employee data is initially loaded from localStorage when the component mounts:

JS

```

useEffect(() => {
  const storedEmployees =
    JSON.parse(localStorage.getItem("employees")) || [];

  setEmployees(storedEmployees);
}, []);
  
```

After the admin updates localStorage:

DIAGRAM

localStorage is updated

But

AllTask employee state
still contains the previously loaded data

For this portfolio implementation, a page refresh is used after an admin adds or deletes a comment.

The refresh flow is:

DIAGRAM

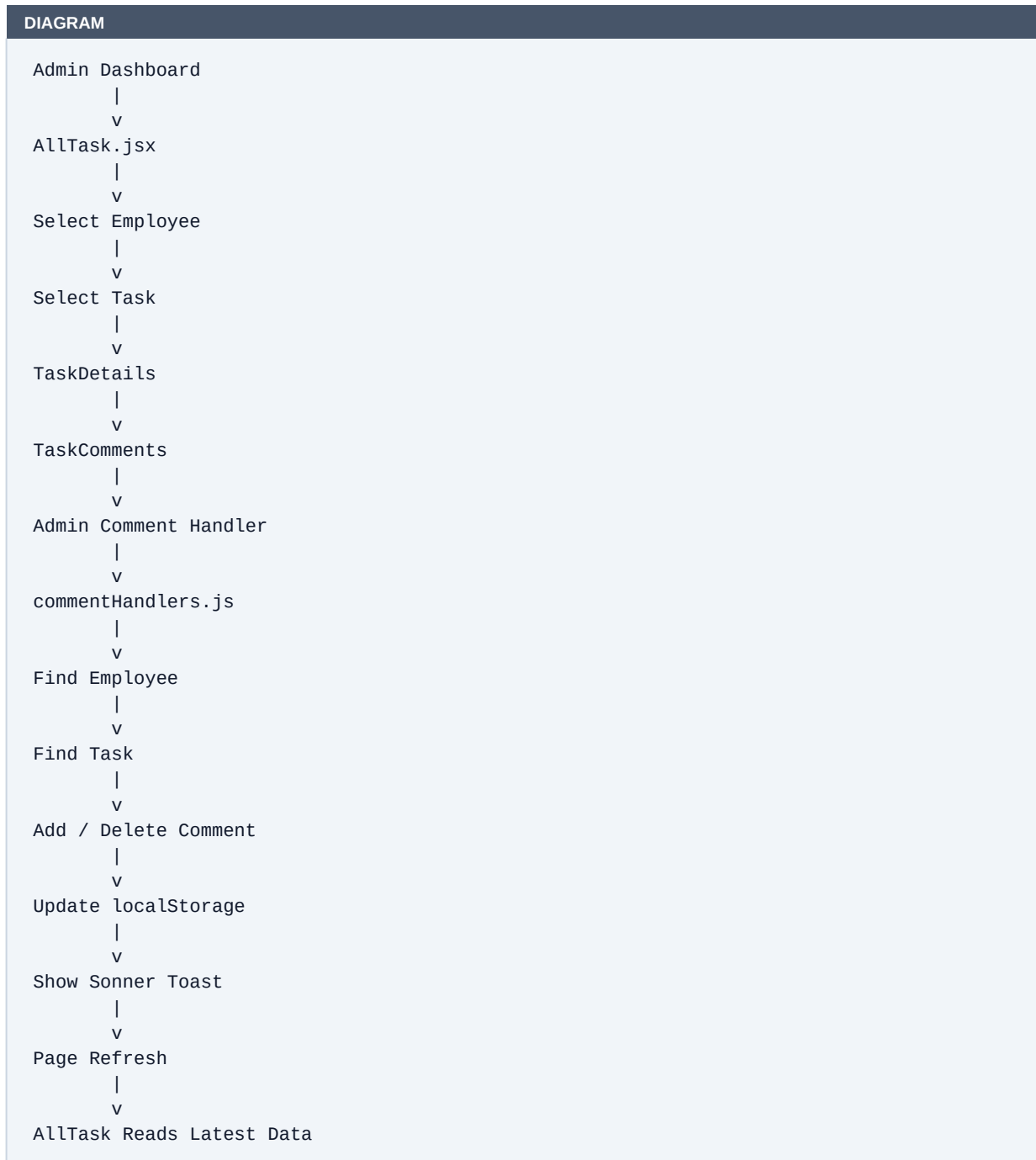
```
graph TD
    A[Admin Updates Comment] --> B[Update employees[] in localStorage]
    B --> C[Show Toast Notification]
    C --> D[Page Refresh]
    D --> E[AllTask Mounts Again]
    E --> F[Read Latest employees[] from localStorage]
    F --> G[Display Updated Comments]
```

This approach keeps the current architecture simple without introducing a larger global state management system.

In a production application, this could later be improved by using shared Context state, Redux, Zustand, React Query, or a backend API with data refetching.

10. Admin Comment Flow

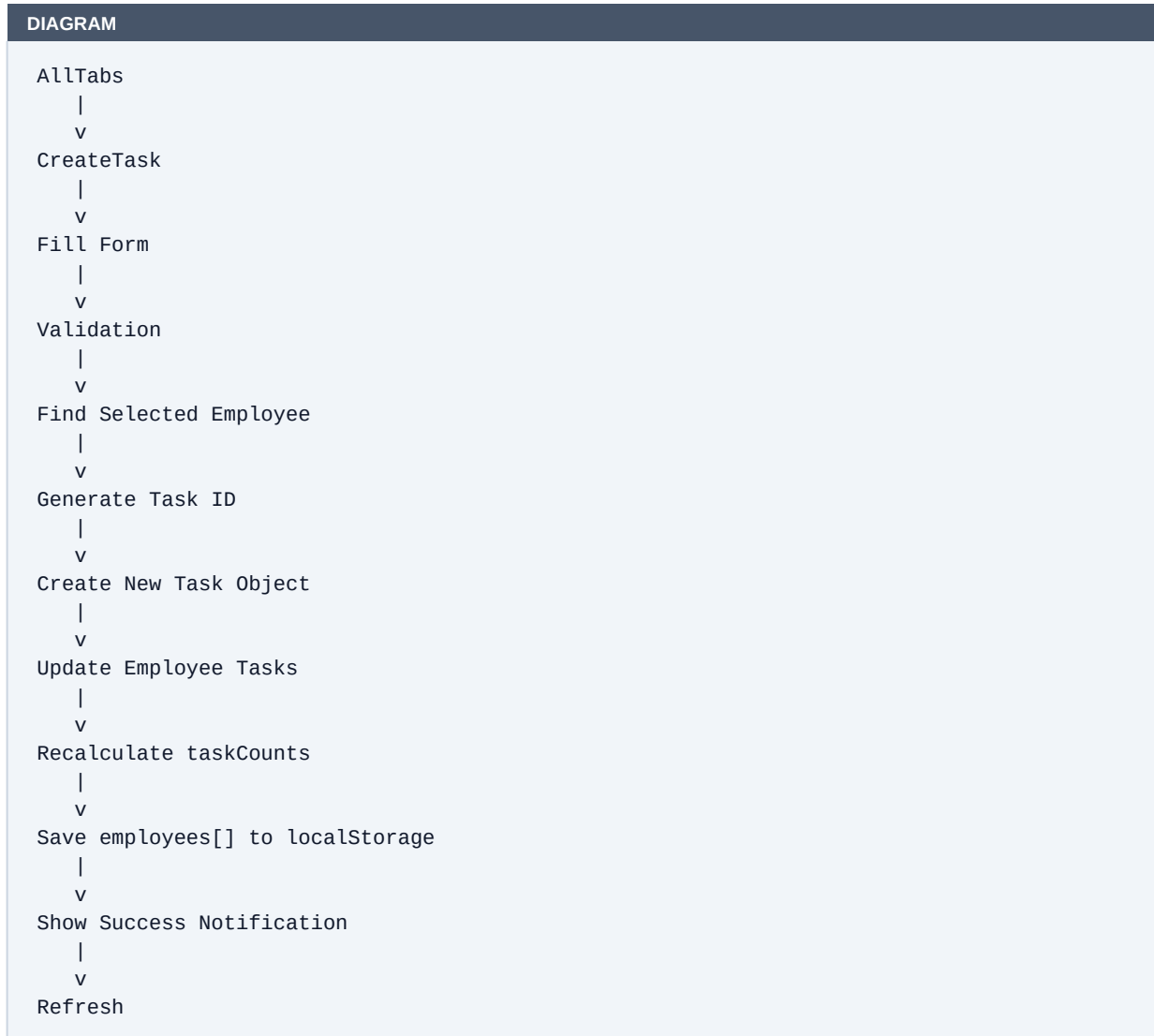
The complete admin comment flow is:



11. Create Task Flow

Task creation is handled inside the admin management section.

The flow is:



A newly created task starts with:

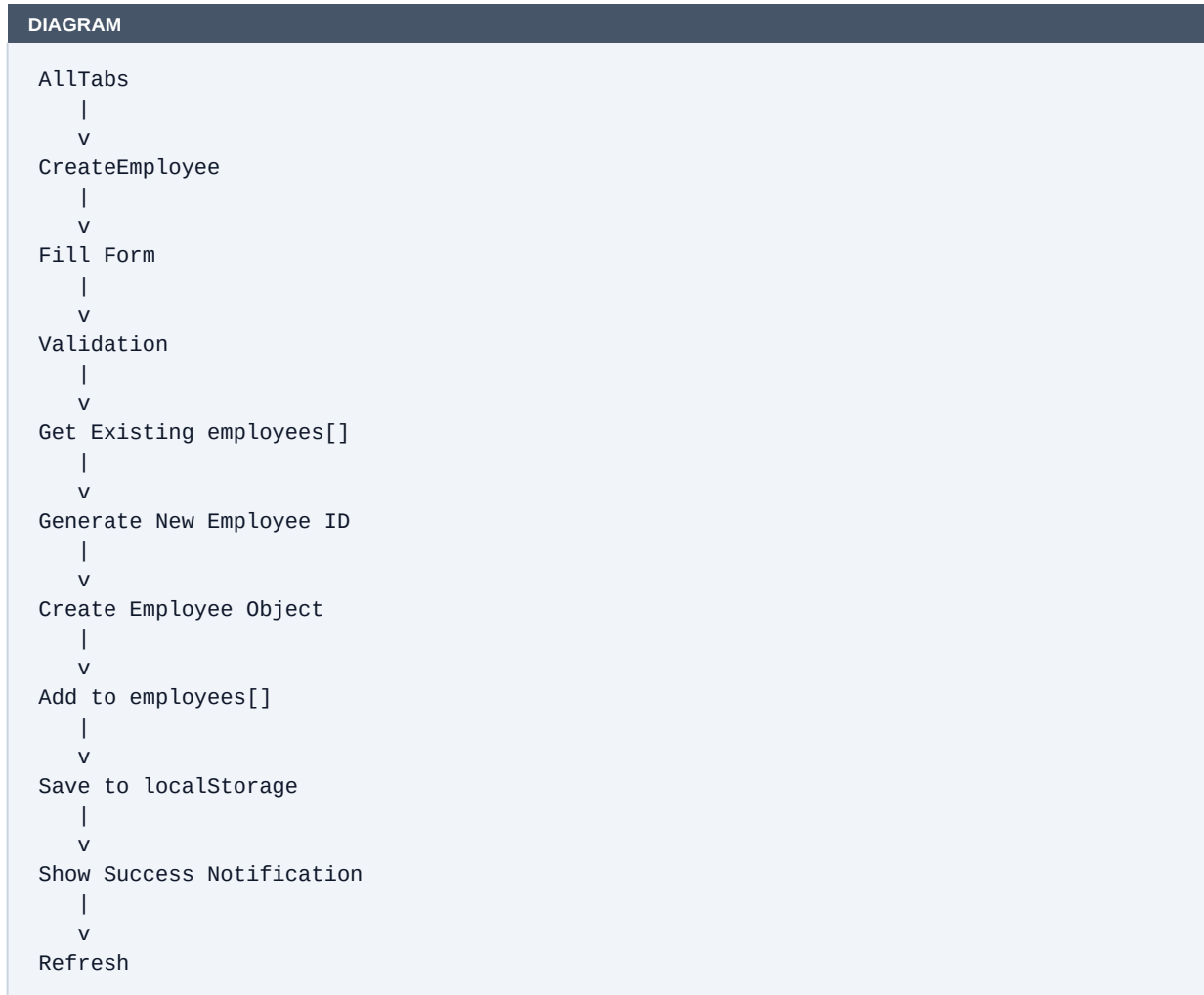
JS

```
{
  active: false,
  newTask: true,
  completed: false,
  failed: false,
}
```

Therefore, every newly created task automatically appears under the New Task status.

12. Create Employee Flow

The employee creation flow works as follows:



Every new employee starts with:

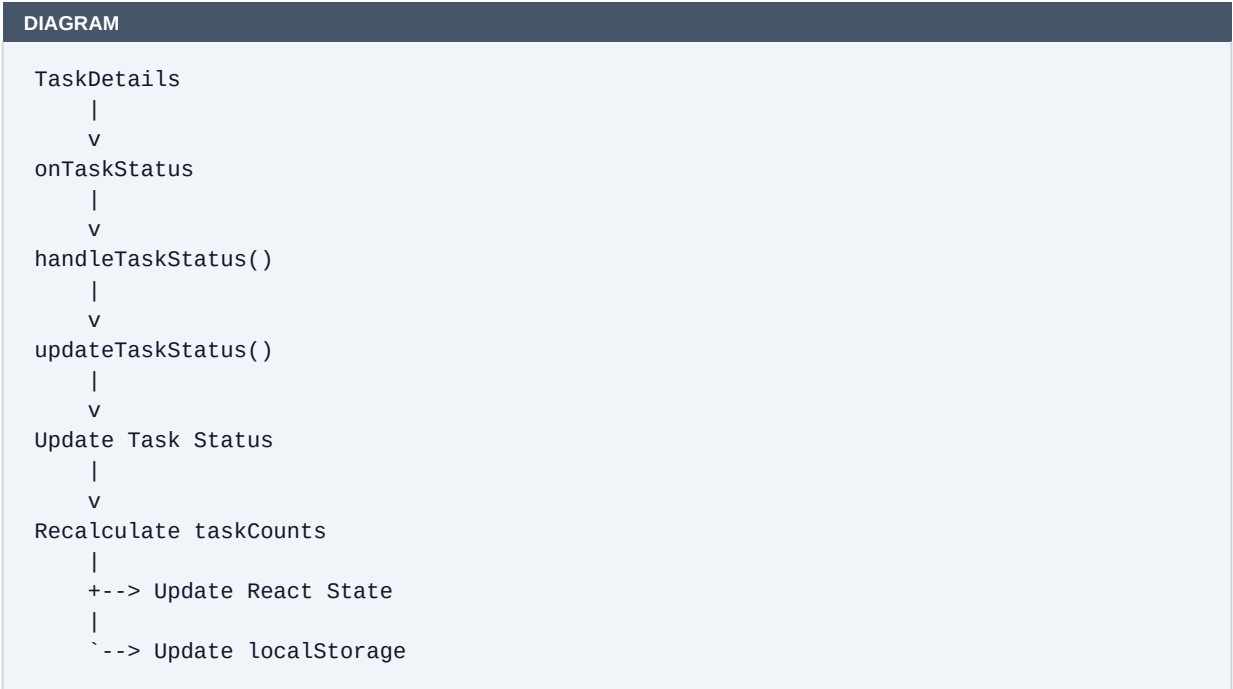
JS

```
taskCounts: {
  active: 0,
  newTask: 0,
  completed: 0,
  failed: 0,
},
tasks: [],
```

Tasks can then be assigned to the employee through the Create Task section.

13. Task Status Flow

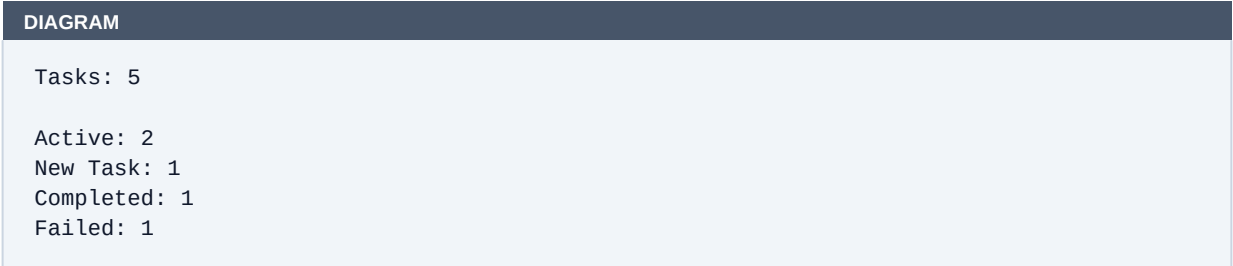
When an employee updates a task status:



The task status helper recalculates the task counts after every status update.

This keeps the task count data synchronized with the actual tasks.

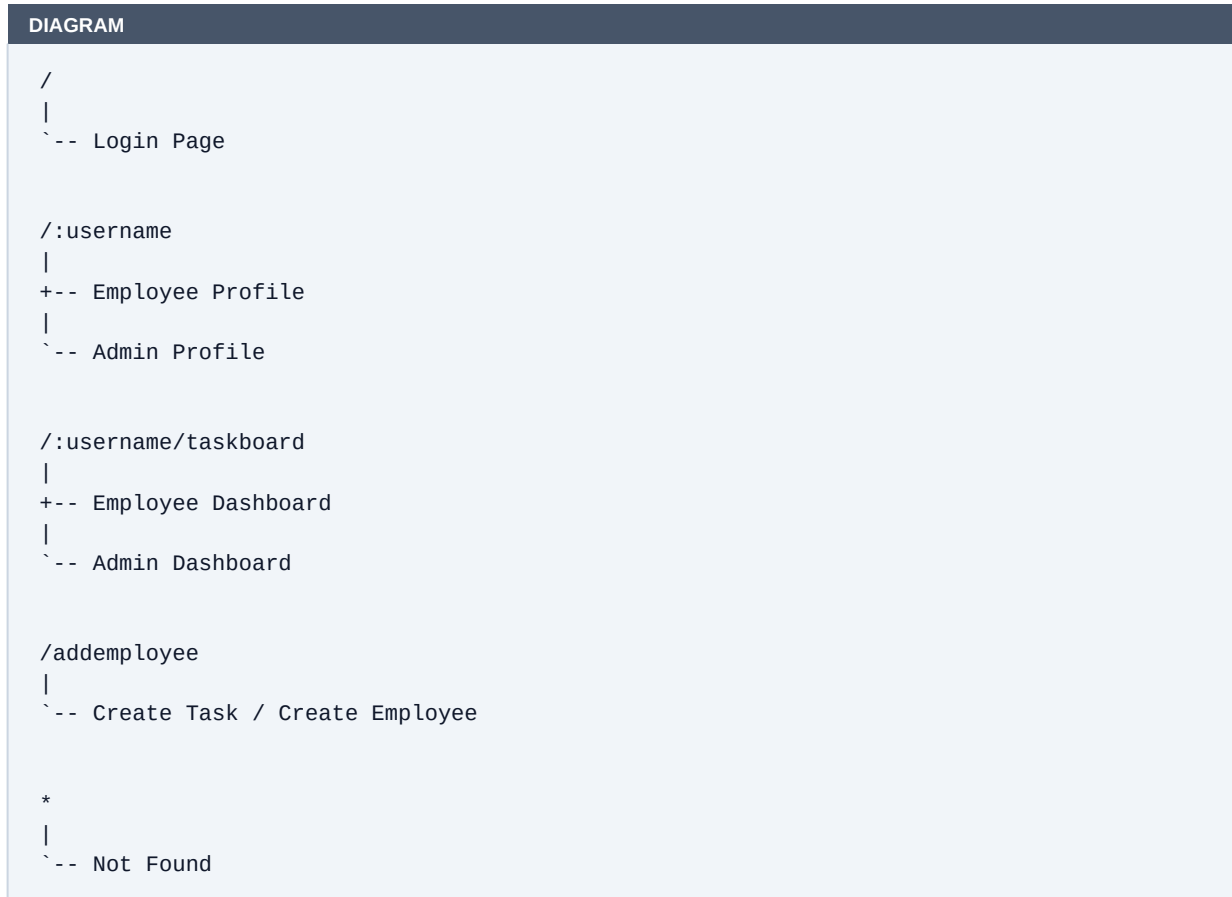
For example:



Total task counts remain consistent with the tasks stored in the employee object.

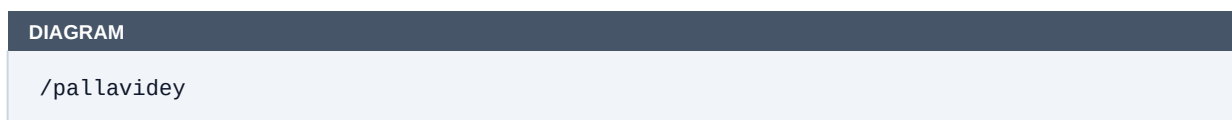
14. Routing Flow

The main application routes are:



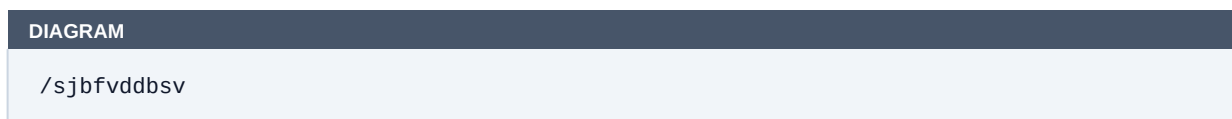
Dynamic routes are validated against the currently logged-in user.

For example:



can display the correct logged-in user's profile.

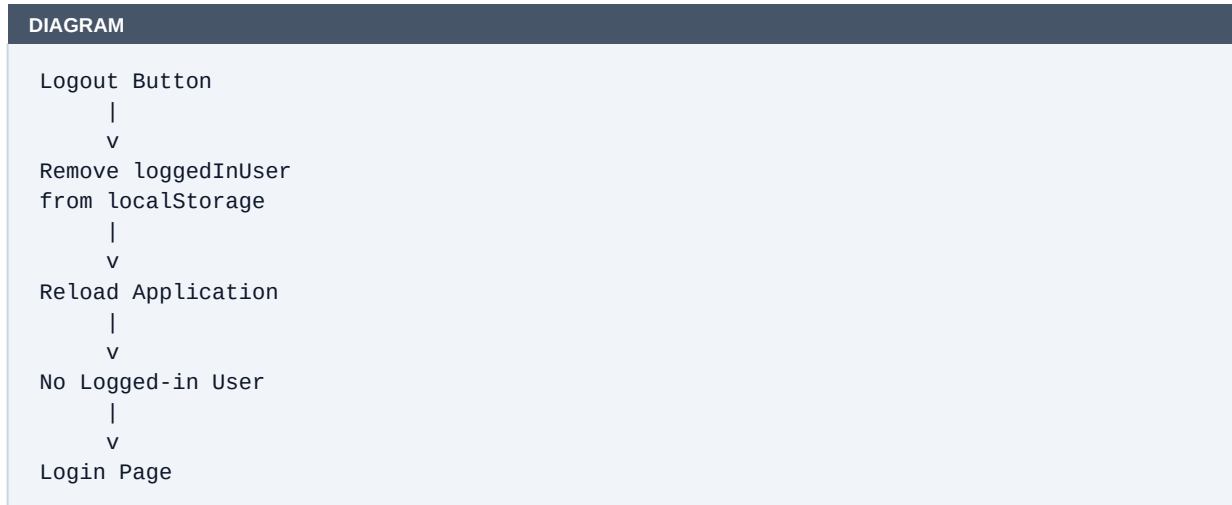
A random URL such as:



displays the Not Found page instead of incorrectly displaying the currently logged-in user's profile.

15. Logout Flow

The logout process works as follows:



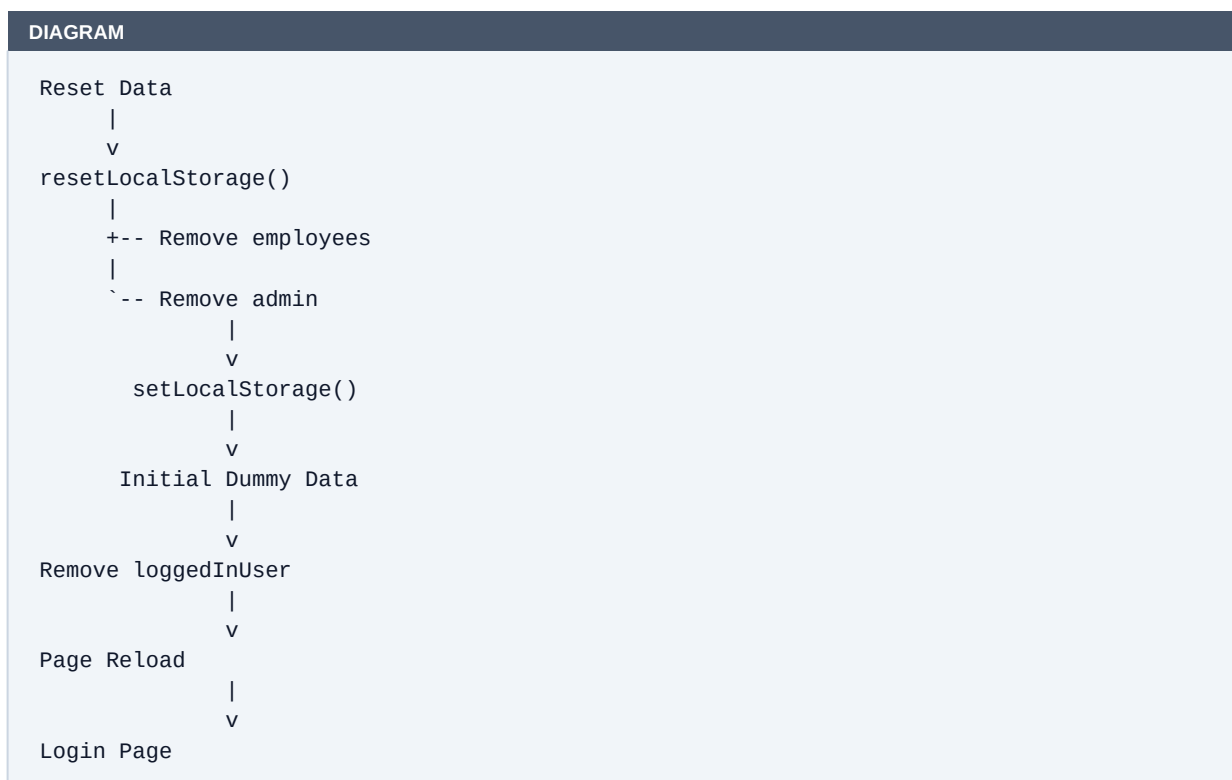
Logging out removes the current session information.

Employee and task data remain stored.

16. Reset Data Flow

Reset Data is different from Logout.

The reset process works as follows:



Logout: Current login session removed. Application data remains.

Reset Data: Current login session removed. Application data reset to initial dummy data.

17. Important Points

Why localStorage is used

localStorage is used as a lightweight client-side persistence layer.

It allows:

- Employee data to persist
- Task updates to persist
- Comments to persist
- New employees to remain available
- New tasks to remain available
- Login sessions to be restored after page refresh

No backend database is required for the current version.

Why React state is still required

localStorage does not automatically update the React UI.

For example, changing `localStorage.setItem(...)` does not automatically trigger a component re-render.

React state is responsible for controlling what appears in the UI.

Therefore, the architecture uses React State to control UI updates, and localStorage to provide data persistence.

Why Context API is used

Context API provides the main employee and admin data throughout the application.

This reduces unnecessary prop drilling and provides a central place for authentication-related data.

Why Employee UI Updates Without Refresh

The logged-in employee's tasks are already stored inside `loggedInUserData`.

After an employee action, `setLoggedInUserData(updatedEmployee)`; updates React state.

React automatically re-renders the relevant UI.

Why Admin Uses a Refresh

The admin object does not contain employee task data.

Admin actions update employee data inside localStorage under `employees[]`.

However, `AllTask` maintains its own employee state loaded when the component mounts.

A page refresh allows the component to mount again and load the latest employee data.

This is a deliberate simplification for the current portfolio implementation.

A future production version could replace this with shared global state or API-based data fetching.

18. Current Technical Coverage

This project demonstrates the following concepts:

React: Functional Components, Props, useState, useEffect, Context API, Conditional Rendering, Dynamic Rendering, Component Communication, State Updates.

React Router: Dynamic Routes, Conditional Routes, Navigation, 404 Handling.

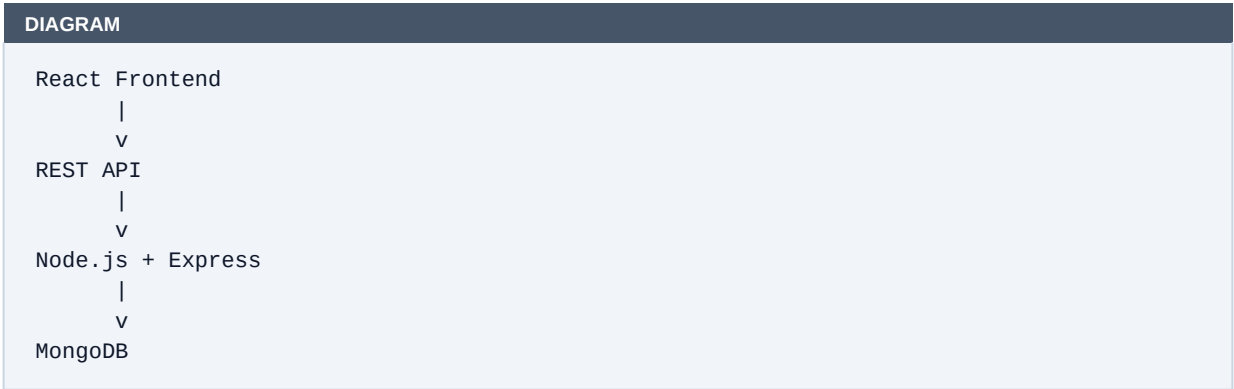
Data Management: localStorage, Data Persistence, Session Restoration, CRUD-style Operations.

Application Features: Admin Role, Employee Role, Authentication Flow, Task Management, Employee Management, Comments, Task Status Updates, Priority Handling, Task Filtering, Toast Notifications, Loading Screen, Reset Functionality.

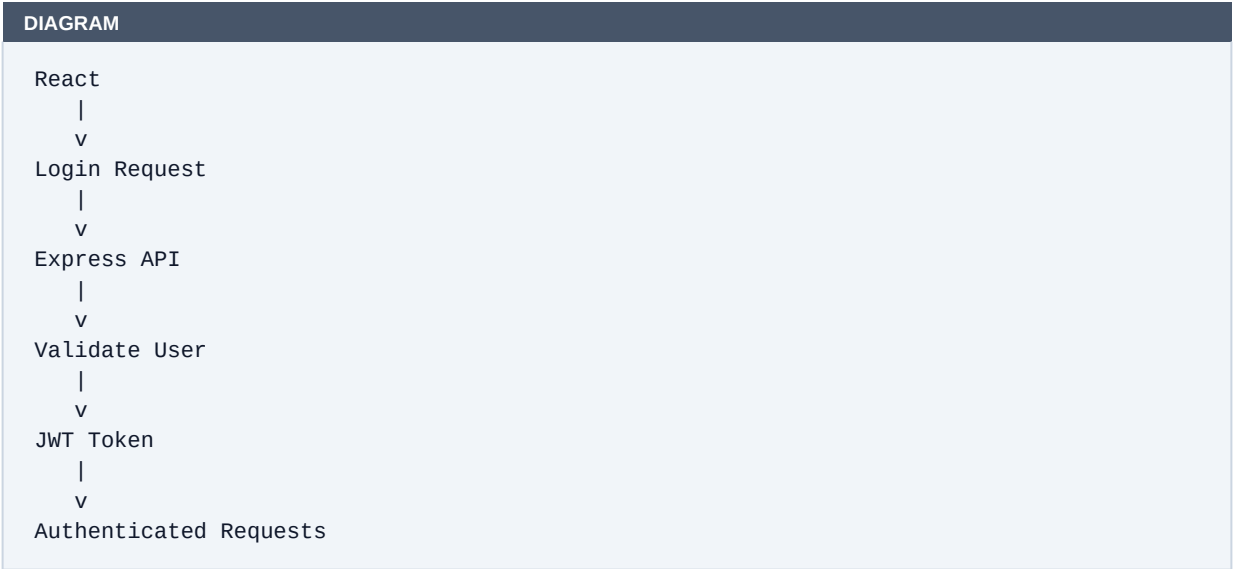
19. Production-Level Direction

The current application is designed as a client-side React project.

A future MERN version could follow this architecture:



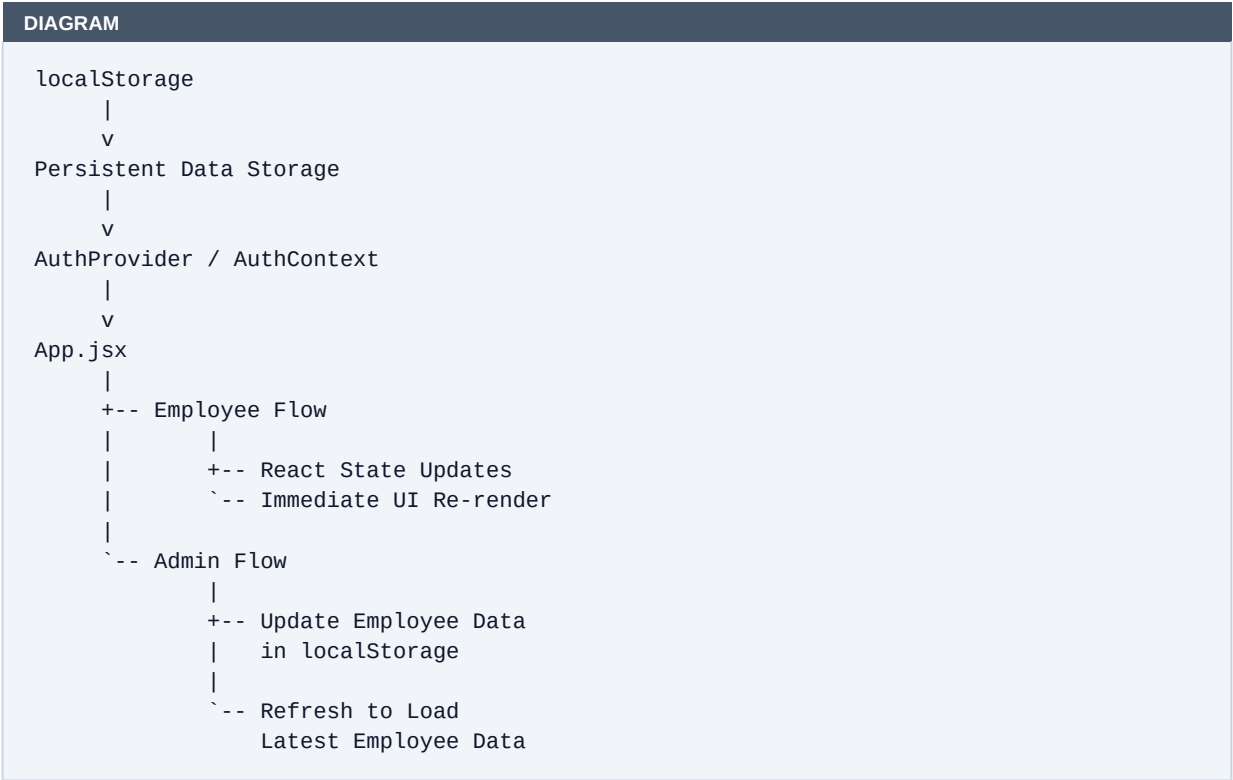
Authentication could be handled using:



The current project provides a strong React foundation for evolving the same application into a full MERN application in the future.

Summary

The Employee Management System uses a combination of React state and localStorage to manage application data.



The application demonstrates role-based functionality, client-side authentication, task management, employee management, data persistence, routing, comments, notifications, and React state management.

The current architecture intentionally keeps the application lightweight and focused on React concepts while providing a clear path for future expansion into a full-stack MERN application.